

Searching Algorithms:

1. Linear Search: Search a target

t=6

4	1	3	6	2	9	7
0	1	2	3	4	5	6

Best case

Worst case

6

Time Complexity

$O(1)$

$O(n)$

Binary Search: Sorted Array $\Rightarrow$ Prerequisite

① t=9

2	3	5	8	9	11
0	1	2	3	4	5

mid = $\frac{s+e}{2}$

$\Rightarrow$ ① arr[mid] == t $\Rightarrow$ return mid

② arr[mid] < t [5 < 9]

s = m+1

s=3 e=5 m=3

m=3

3 2
1 2 7

t=3

2	3	5	8	9	11
0	1	2	3	4	5

① arr[mid] == t

② arr[mid] < t

s < 3

③ arr[mid] > t

e = m-1

m = (s+e)/2

m = 2

* Sq root

* Peak in a mountain

s e
s e

```

{
    s = 0;
    e = n-1;
    m = (s+e)/2;
    while (s <= e) {
        if (arr[mid] == key) {
            return mid;
        }
        else if (arr[mid] < key) {
            s = mid+1;
        }
        else {
            e = mid-1;
        }
        mid = (s+e)/2;
    }
    return -1;
}
    
```

0

e

2 2

s

0

s > e

key

s mid e

key

s mid e

Recursive Binary Search:

* int binarysearch-recurson (arr, n, s, e, k)

{

① arr[mid] == key

return mid

② arr[mid] < key

return bs (arr, n, mid+1, e, k)

③ arr[mid] > key

return bs (arr, n, s, mid-1, k)

}

return -1;

Binary Search Mid Threshold Formula

Optimal Binary Search Mid Formula

mid = $\frac{\text{INT-MAX} + \text{INT-MAX}}{2}$ = (-) ve < 0 to n-1 >

[mid = $\frac{s+e}{2}$] TLE

[mid = $s + \left(\frac{e-s}{2}\right)$]

= $\frac{2s + e - s}{2}$ = $\frac{s+e}{2}$

Applications of Binary Search:

1. First & Last Occurrence of an element in an array using Binary Search. ***

arr = 1, 2, 3, 3, 3, 3, 4, 5

First Occurrence

Last Occurrence

3

FO $\rightarrow$ 2

LO $\rightarrow$ 5

arr = -1;

if arr[mid] == target

ans = mid;

e = mid-1;

arr[mid] > target

e = mid-1;

arr[mid] < target

s = mid+1;

return ans;

s = 0

e = 7

= $\frac{7}{2}$

= 3

t t

s mid e

1 2 3 3 3 3 4 5

0 1 2 3 4 5 6 7

fo lo

B.S

Sorted Array

4

sort()

fo $\rightarrow$ 2

lo $\rightarrow$ 5

total number of occurrences: ?

to = lo - fo + 1

= 5 - 2 + 1

= 3 + 1 = 4

Time Complexity of Binary Search:

k is constant $\Rightarrow$

0, 1, 2, 3, ...

In Logarithmic

Equation:

k = $\log_2 N$

Time Complexity

$O(\log N)$

N

N

N

N

N

N

N

N

N

N

N

N

N

N

$2^0 = 1$

$2^1 = 2$

$2^2 = 4$

$2^3 = 8$

$2^k = 0, 1, 2, 3$